

Vektor Official Program Document

Authoritative internal build and operations document for taking Vektor from the current Phase Alpha state to the final local-first, vendor-backed, free-VM-hosted, Vercel-distributed system.

As of: 2026-04-23 (America/Phoenix) · Repository: TradingBot · Current Phase Alpha Progress: 79 / 100

1. Executive Summary

Vektor is being built as a local-first, AI-native hedge fund operating system. The final target state is a system where the CEO interacts with Vektor through OpenClaw and admin surfaces, backend workers run continuously on a persistent free VM, specialist agents use hosted vendor APIs rather than weak local models, and public distribution surfaces run on Vercel. Every report, trade, approval, model score, and publication must be traceable.

2. Current State Snapshot

2.1 What is already real

- Backend orchestration, task bus, approvals, digests, audit trail, and admin war room are live.
- Hosted multi-vendor routing exists with cooldowns, failover, concurrency caps, request envelopes, and deferred retries.
- Strict real-data mode and paper-only execution are enforced.
- Multi-asset paper execution routes exist for equities, options, forex, and crypto intents.
- Discovery scoring, ML scoring, and adaptive ML gating now affect actual trade control.
- CEO approval workflow exists for trades, allocation changes, public publication, and major reroutes.
- Performance tracking, knowledge lineage, and local soak tooling already exist.

2.2 What is still incomplete

- Multi-day local soak has not yet been completed and signed off.
- Free-VM deployment hardening is not complete.
- Discovery and world scanning still need higher-quality ranking and pruning.
- ML still needs empirical threshold tuning from realized outcomes.
- Risk still needs deeper thesis degradation and post-trade review behavior.

3. Phase Alpha: Backend Fund OS

3.1 Objective

Make Vektor real as a backend-native fund operating system before shifting attention to broader platformization or external polish.

3.2 Scope

- Canonical CEO/Vektor/OpenClaw command layer
- Backend-native worker orchestration and swarm execution
- Hosted LLM routing with vendor redundancy
- Strict real-data-only operation
- Paper broker execution under policy and risk gates
- Sleeve and asset-class capital allocation controls
- Market scanner and world-event discovery
- ML scoring and confidence estimation
- Approval workflows and digests
- Local-first runtime reliability and soak readiness

3.3 What Vektor should be able to do after Alpha

- Answer CEO questions about positions, exposure, risk, winners/losers, and pending approvals.

- Scan what to trade, decide not to trade when conditions are weak, run swarms, request approvals, and paper-execute safely.
- Maintain a traceable chain from signal to research to decision to trade to attribution.

3.4 Alpha remaining checklist

- Complete a real 48-72 hour local soak with monitoring.
- Confirm recovery after restart, halt, and deferred queue pressure.
- Improve discovery ranking and world-scanning quality.
- Finish ML outcome feedback and threshold tuning loop.
- Finish deeper risk behavior around thesis degradation and post-trade review.
- Finalize free-VM hardening after local soak success.

Alpha exit rule: Alpha is only complete when Vektor survives a clean 48-72h local soak, keeps approval/gating flows stable under provider pressure, shows credible no-trade behavior, and is ready to move to the VM without using cloud deployment as a debugging shortcut.

4. Phase Beta: Data, Memory, and Knowledge Infrastructure

4.1 Objective

Turn Vektor into a persistent memory system that stores and references every meaningful internal artifact.

4.2 Scope

- Canonical document storage for reports, tasks, decisions, approvals, and model outputs
- Durable retrieval by run, symbol, agent, asset class, and lineage
- Graph projection layer built from canonical storage
- Reusable research memory and knowledge retention policies

4.3 What Vektor should be able to do after Beta

- Answer with provenance, reconstruct why a trade happened, and show what each agent found, who used it, and what output it changed.
- Use past work as real working memory rather than ad hoc context.

5. Phase Gamma: Operator OS and Internal CRM

5.1 Objective

Make admin the real operating system for the fund rather than a monitoring dashboard.

5.2 Scope

- Dense admin command center with per-agent process visibility
- Staged wave inspection, pack inspection, and approval operations
- Allocation control, intervention controls, market watch, and audit/performance drill-downs
- Information architecture that is operationally distinct and non-duplicative

5.3 What Vektor should be able to do after Gamma

- Show exactly what every agent is doing right now.
- Let the CEO pause, reroute, retry, cancel, approve, and recover from one control plane.

6. Phase Delta: Research and Publication Engine

6.1 Objective

Create a publication system that turns internal work into high-quality, approval-gated public content.

6.2 Scope

- Internal-only by default for generated content
- Explicit promotion workflow to public research and public blog

- Editorial review queue and approval loop
- blog-next as the public blog distribution surface

6.3 What Vektor should be able to do after Delta

- Draft market briefs, case studies, and research papers from actual internal evidence and publish only approved content.

7. Phase Epsilon: Persistent Infrastructure and Expansion

7.1 Objective

Move from local-only operation to persistent remote runtime without sacrificing control or reliability.

7.2 Scope

- Persistent free cloud VM backend deployment with static IP
- Supervised startup, reboot recovery, health checks, log rotation, and backups
- OpenClaw remains on the laptop as CEO interface
- Public landing/blog/research surfaces run on Vercel

7.3 What Vektor should be able to do after Epsilon

- Run continuously in paper mode on a free VM while remaining locally controlled through OpenClaw/Discord and admin.

8. Phase Zeta: Final Operating Standard

8.1 Objective

Bring Vektor to a hardened, investor-facing, externally defensible operational standard.

8.2 Scope

- Investor-grade reporting, compliance preparation, and policy evidence
- Security hardening and mature secret management
- Stable vendor abstraction for paid data and stronger model providers
- Empirical tuning loops from real paper-trading outcomes

8.3 What Vektor should be able to do after Zeta

- Operate as a credible, traceable, local-first, vendor-backed fund OS ready for external review.

9. Final Target Architecture

- **CEO layer:** CEO interacts with Vektor through OpenClaw/Discord and admin using natural-language queries and explicit commands.
- **Runtime layer:** backend workers run continuously on a free VM; orchestrator, task bus, policy gate, approval center, risk engine, and execution adapters live there.
- **Memory/data layer:** SQL is canonical; graph is a projection.
- **Public layer:** Vercel hosts the landing site, public blog, and public research surfaces from approved backend content only.

10. Recommended Immediate Execution Order

- Run the real 48-72h local soak.
- Review soak logs for queue, provider, and recovery failures.
- Tighten remaining discovery, risk, and ML gaps found by soak.
- Only then complete free-VM deployment hardening.
- Continue Beta and Gamma work without breaking Alpha reliability.

11. Final Standard for Vektor

When the full program is complete, Vektor should be able to act as the canonical fund control authority for the CEO, discover opportunities, reject weak ones, request approvals on strong ones, coordinate specialist swarms under hosted vendor routing, enforce capital and risk

discipline, persist and explain every decision, run continuously on a free VM while public surfaces are served through Vercel, and generate a defensible track record with supporting documentation for real external review.

This document is intended to be the authoritative internal roadmap and operating specification for the remaining buildout of Vektor from the current Phase Alpha state to the final local + vendor + free-VM + Vercel architecture.